# ความคืบหน้างาน SHAP (Cross-Dataset) แบบแบ่ง 70/30

รันงาน SHAP ใหม่ทั้งหมดบน split train 70% / test 30% (ฉบับก่อนใช้ train 60% ที่กัน calibration 10% ไว้แต่ไม่เคยใช้) ตัวเลขผลทุกตัวในเอกสารนี้อ่านจากไฟล์ผลที่รันใหม่ ส่วนที่เป็นขั้นตอนและเหตุผลการออกแบบ (หัวข้อ 2 ถึง 6, 9, 10) คงไว้ตามเดิม

## ผลที่เปลี่ยนเมื่อแบ่ง 70/30

สคริปต์ เกณฑ์ที่เขียนไว้ก่อนรัน (ไฟล์ pre-registration) และ seed เหมือนเดิมทุกอย่าง
สิ่งเดียวที่เปลี่ยนคือสัดส่วนการแบ่ง ผลของ Part B ทุกรอบใช้ split ของโปรโตคอล cross-dataset จึงเปลี่ยนตาม ส่วน Part A ใช้การแบ่งสุ่ม 70/30 ภายในของตัวเอง (`train_test_split`) ไม่ได้ใช้ split ที่แก้
ผลของมันจึงเท่าเดิมเป็นส่วนใหญ่ (ค่าที่ขยับเล็กน้อยไม่ได้มาจาก split เพราะ Part A ไม่ใช้ split นั้น น่าจะเป็นความไม่คงที่ของการเทรน แต่ยังไม่ได้ทดสอบ)

ตัวที่ทำให้ผลเปลี่ยนคือ SSH-Patator ของ 2017 ที่ train 60% ไม่มีเลย (0 แถว) และที่ train 70% มี 474 แถว โมเดลที่เทรนจาก 2017 จึงทาย Brute Force ใน 2018 ถูกบางส่วน และ Brute Force กลายเป็นคลาสที่มีแถว caught ให้เทียบ

| ขั้น | train 60% (ฉบับเดิม) | train 70% (ฉบับนี้) |
|---|---|---|
| รอบ 1 Part A kappa (LightGBM / LR) | 0.366 / 0.334 | 0.366 / 0.334 (เท่าเดิม) |
| รอบ 1 Part B ตาราง SHAP | partial (3 คลาส) | partial (4 คลาส) |
| รอบ 1 Part B ตาราง permutation | ไม่สนับสนุน (3 คลาส) | partial (4 คลาส) |
| รอบ 2 (กลุ่มหลัก consensus) | ไม่สนับสนุน (3 คลาส) | ไม่สนับสนุน (4 คลาส) |
| รอบ 3 กลุ่มหลัก (3 โมเดล) | ไม่สนับสนุนทั้ง 3 | ไม่สนับสนุน (3 จาก 3 โมเดล) |
| รอบ 3 กลุ่ม SHAP (26) รายคลาส | ไม่มีคลาสที่ผ่าน | ผ่าน 2 คลาส (xgboost DoS (G = +0.228, p = 0.0045) และ catboost DDoS (G = +0.246, p = 0.0090)) ภาพรวมของโมเดลยังไม่สนับสนุน |
| within-target (11 กรณีที่นับได้) | ไม่ผ่านทั้ง 9 กรณี | ไม่ผ่านทั้ง 11 กรณี |

ข้อสรุปหลักไม่เปลี่ยน คือไม่มีหลักฐานว่ากลุ่ม feature ที่บอก dataset (F) เป็นสาเหตุให้การโจมตีถูกทายเป็น Benign ส่วนที่ต้องระบุคือ 2 กรณีรายคลาสของกลุ่ม SHAP 26 ตัวในรอบ 3 ที่ผ่านเกณฑ์ Bonferroni ซึ่งฉบับ 60% ไม่มี กลุ่มนี้ไม่ใช่กลุ่มตัดสินหลัก ภาพรวมของทั้งสองโมเดลยังไม่สนับสนุน และเมื่อเทียบใน dataset ปลายทางเดียวกัน (within-target) XGBoost DoS ได้ G_within = +0.073 (p = 0.21) ซึ่งไม่ผ่าน ส่วน CatBoost DDoS ตรวจใน within-target ไม่ได้ เพราะโมเดลทาย DDoS ใน 2018 ถูก 0 แถว

## 1. สถานะตอนนี้

| ขั้น | สถานะ |
|---|---|
| เขียนเกณฑ์ล่วงหน้า (pre-registration) | เสร็จ (ไม่แก้ในการรันใหม่) |
| เขียนสคริปต์ | เสร็จ |
| ทดสอบโค้ดกับข้อมูลชุดเล็ก | ผ่าน (รันซ้ำก่อนรันจริงบน 70/30 ยืนยันว่า log ขึ้น train 14,000 / test 6,000 จากตัวอย่าง 20,000) |
| รันจริง รอบ 1 | เสร็จ (ใช้เวลา 15.1 นาที) |

| ขั้น | สถานะ |
|---|---|
| ตรวจความทนทาน (เกณฑ์ และโหมด SHAP) | เสร็จ (ข้อ 11, 12) |
| Part B รอบ 2 พร้อม null | เสร็จ |
| รอบ 3 กับ RF, XGBoost, CatBoost | เสร็จ (รันขนานแยกโมเดล XGBoost 51 นาที CatBoost 75 นาที RF 650 นาที) |
| within-target | เสร็จ (รายงานแยก `shap_within_target.md`) |
| สรุปผล | Part A ไม่ยืนยันตารางเดิม, Part B รอบ 2 และรอบ 3 ไม่สนับสนุนในกลุ่มหลัก (ดูข้อ 13, 14, 16, 17) |

## สรุปสั้น

1. SHAP ให้ตารางบทบาทของ feature ต่างจาก permutation พอสมควร (kappa 0.37 และ 0.33 ต่ำกว่าเกณฑ์ 0.40) SHAP พบ feature ที่บอก dataset ได้ 26 ตัว ส่วน permutation พบ 18 ตัว (Part A ไม่ขึ้นกับ split ที่แก้ จึงเท่าเดิม)
2. feature ที่เปลี่ยนกลุ่มประมาณครึ่งหนึ่งมี feature อื่นที่สหสัมพันธ์สูง (|r| >= 0.9) ซึ่งตรงกับจุดอ่อนของ permutation ที่คาดไว้
3. รอบ 1: คำถามว่า "การทายพลาดหลังย้าย dataset เกิดจาก fingerprint หรือไม่" ได้ partial ทั้งสองตาราง (ตาราง SHAP สนับสนุน 1 คลาส ไม่สนับสนุน 2 กลาง ๆ 1 จาก 4 คลาส ตาราง permutation สนับสนุน 0 ไม่สนับสนุน 2 กลาง ๆ 2) ฉบับ 60% ตาราง permutation ได้ไม่สนับสนุน ที่เปลี่ยนเพราะ Web Attack ในตาราง permutation ขยับจาก -0.098 เป็น +0.023 และ Brute Force เข้ามาเป็นคลาสที่ตัดสินได้
4. รอบ 2 (เพิ่ม null จากการสุ่มกลุ่ม feature): ไม่สนับสนุน ทุกกลุ่ม feature ส่วนต่างที่เห็นในรอบ 1 เช่น DDoS +0.144 อยู่ในช่วงที่กลุ่ม feature แบบสุ่มขนาดเท่ากันก็ให้ได้ (p = 0.13) จึงยังไม่มีหลักฐานว่า feature ที่บอก dataset เป็นตัวผลักให้ทายพลาดเป็น Benign
5. ตรวจความทนทาน: การเปลี่ยนโหมด SHAP แทบไม่มีผล (Spearman 0.98) ส่วนความต่างของ Part A มาจากเกณฑ์ตัดสินส่วนหนึ่ง (ใช้ top-K เดียวกัน kappa ขึ้นเป็น 0.40 ถึง 0.62)
6. รอบ 3 (RF, XGBoost, CatBoost ด้วยวิธีของรอบ 2): ไม่สนับสนุน ทั้ง 3 โมเดลในกลุ่มหลัก ผลของรอบ 2 จึงไม่ได้เกิดกับ LightGBM ตัวเดียว ในกลุ่ม SHAP 26 ตัวมี 2 คลาสที่ผ่านเกณฑ์รายคลาส (xgboost DoS (G = +0.228, p = 0.0045) และ catboost DDoS (G = +0.246, p = 0.0090)) แต่ภาพรวมของโมเดลยังไม่สนับสนุน
7. within-target (เทียบแถวที่ทายพลาดกับที่ทายถูกใน dataset ปลายทางเดียวกัน): ไม่ผ่านทั้ง 11 กรณีที่นับได้ แปลว่า gap ที่เห็นในรอบ 2-3 มาจากการต่าง dataset ไม่ใช่จากการทายผิด

## 2. ทำไมต้องทำ SHAP

งานก่อนหน้า (`dataset_vs_attack_importance.py`) แบ่ง feature 60 ตัวเป็น 4 กลุ่ม คือ fingerprint only, attack only, entangled และ neither โดยใช้ permutation importance

permutation importance มีจุดอ่อนเรื่อง feature ที่ซ้ำกัน ถ้าสอง column มีข้อมูลเดียวกัน สลับค่า column หนึ่งแล้วโมเดลก็ยังอ่านอีก column ได้ ทำให้ทั้งสองตัวได้คะแนนใกล้ 0 ผล adversarial validation บอกแล้วว่าสัญญาณที่บอก dataset กระจายอยู่หลาย column จึงเป็นไปได้ว่าตารางบทบาทของ feature ได้รับผลจากจุดอ่อนนี้

SHAP แบ่งคะแนนให้ column ที่ซ้ำกันแทนที่จะให้ 0 ทั้งคู่ และอธิบายได้รายแถว งานนี้จึงมี 2 ส่วน

- Part A ตรวจว่าตารางบทบาทของ feature ยังเหมือนเดิมไหมเมื่อใช้ SHAP
- Part B ดูการโจมตีที่โมเดลทายพลาดเป็น Benign หลังย้าย dataset ว่าถูกผลักด้วย feature กลุ่มไหน

## 3. เกณฑ์ที่ตั้งไว้ก่อนรัน

ไฟล์: `results/crossdataset/importance_comparison/SHAP_PREREGISTRATION.md`

Part A

- ใช้ข้อมูลและการแบ่งเหมือนงาน permutation ทุกอย่าง (60 features, seed 42-46, 70/30)
- โมเดล LightGBM และ Logistic Regression
- feature ที่ถือว่า "ใช้งาน" ต้องมีส่วนแบ่ง SHAP อย่างน้อย 1/60 และเป็นแบบนั้นใน 4 จาก 5 seeds
- เทียบกับตารางเดิมด้วย Cohen's kappa
- kappa >= 0.60 ทั้งสองโมเดล = ยืนยันตารางเดิม
- kappa < 0.40 โมเดลใดโมเดลหนึ่ง = ไม่ยืนยัน
- นอกนั้น = partial

Part B

- LightGBM แบบเดียวกับ protocol (77 features, 7 classes, แบ่งตามเวลา) ทิศหลักคือ 2017 ไป 2018
- missed = การโจมตีใน test ของอีก dataset ที่ถูกทายเป็น Benign
- caught = การโจมตีคลาสเดียวกันใน test ของ dataset เดิมที่ทายถูก
- วัดว่าแรงที่ผลักไปทาง Benign มาจาก feature กลุ่ม fingerprint only หรือ entangled ที่ส่วน (ค่า s)
- ถ้า s(missed) - s(caught) >= 0.10 ทุก seed ในคลาสส่วนใหญ่ = fingerprint ทำให้ทายพลาด
- ถ้าต่างกันไม่เกิน 0.02 ในคลาสส่วนใหญ่ = ไม่สนับสนุน
- นอกนั้น = partial

## 4. สิ่งที่ทำไปแล้ว

1. ติดตั้ง `shap` 0.52.0 ใน `.venv` (มีใน `requirements.txt` แต่ยังไม่ได้ติดตั้ง)
2. เขียน `scripts/shap_importance.py` ใช้ฟังก์ชันเดิมจากสคริปต์ของเพื่อน (`dataset_vs_attack_importance.py`, `adversarial_validation.py`, `run_crossdataset.py`) เพื่อให้ข้อมูล การแบ่ง และโมเดลเหมือนกันทุกอย่าง
3. ทดสอบโค้ดกับข้อมูลชุดเล็ก (1 seed, 20,000 แถวต่อ dataset) ใช้ไฟล์ตัวอย่าง 2018 ที่ทำความสะอาดเองเฉพาะสำหรับทดสอบ ตัวเลขจากการทดสอบนี้ ไม่ใช่ผลการทดลอง ใช้ดูแค่ว่าโค้ดทำงานถูก

## 5. สิ่งที่เจอระหว่างทดสอบ และสิ่งที่แก้

ทุกข้อแก้ก่อนรันจริง และบันทึกไว้ในไฟล์เกณฑ์แล้ว

| เรื่องที่เจอ | ที่แก้ |
|---|---|
| SHAP แบบ interventional ของ LightGBM ใช้เวลา 9 นาทีต่อ 1 ครั้ง ถ้ารันครบจะใช้ประมาณ 20 ชั่วโมง | LightGBM เปลี่ยนเป็นแบบ tree_path_dependent ทดสอบทั้งหมดเหลือ 1.4 นาที |
| `shap` ลด background จาก 200 เหลือ 100 แถวเองโดยไม่บอก | กำหนด `max_samples` ให้ใช้ครบ 200 (Logistic Regression) |
| Part B เอาคลาส Infiltration มาตัดสินทั้งที่ฝั่ง caught มีแค่ 10 แถว | เพิ่มเกณฑ์ caught อย่างน้อย 30 แถว (ค่าเดียวกับ protocol) |
| เกณฑ์ฝั่ง attack ยังไม่ได้บอกว่านับ dataset ไหน | ใช้ "dataset ใดก็ได้" เหมือนงาน permutation |
| จำกัดแถวใน Part B ที่ 2,000 ต่อกลุ่ม ทำให้คลาสเล็กแทบไม่เหลือ | เปลี่ยนเป็น 1,000 แถวต่อคลาส |

เรื่องการเปลี่ยนเป็น tree_path_dependent: ตอนแรกตั้งใจใช้ interventional ทั้งสองโมเดล
เพื่อให้วัดแบบเดียวกัน แต่ kappa ของแต่ละโมเดลเทียบกับตาราง permutation ของโมเดลตัวเอง
การที่สองโมเดลใช้คนละแบบจึงไม่กระทบการตัดสินของ Part A

## 6. การรันจริง

- ข้อมูล 2018 เต็มชุดได้จากเพื่อน (`cicids2018_clean.parquet` 11,445,955 แถว สร้างจาก commit `cb5361c`) วางไว้ที่ `data/ids2018/`
- สุ่มมา dataset ละ 300,000 แถว, 60 features, seed 42-46
- ก่อนรัน สคริปต์บันทึก sha256 ของไฟล์เกณฑ์ไว้ใน `summary.json` (`8f701b15...f85173`) ถ้ามีการแก้เกณฑ์หลังเห็นผลจะตรวจได้
- ผลอยู่ที่ `results/crossdataset/importance_comparison/shap/`

หมายเหตุ: ผลที่อธิบายในหัวข้อนี้เป็นการรันครั้งแรกบน train 60% ผลทั้งหมดถูกแทนที่ด้วยการรันใหม่บน 70/30 ส่วนตัวเลขการรันครั้งแรกอยู่ในประวัติ git

## 7. ผล Part A: ตารางบทบาทของ feature

คำตัดสินตามเกณฑ์: ไม่ยืนยัน (kappa ต่ำกว่า 0.40 ทั้งสองโมเดล)

| | LightGBM | Logistic Regression |
|---|---|---|
| Cohen's kappa | 0.366 | 0.334 |
| Spearman ของอันดับ attack importance (2017 / 2018) | 0.68 / 0.69 | 0.74 / 0.88 |
| feature ที่เปลี่ยนกลุ่ม | 25 จาก 60 | 29 จาก 60 |
| ในนั้นมีคู่ที่สหสัมพันธ์ >= 0.9 | 13 | 16 |
| ในนั้นเป็นตัวแทนกลุ่ม feature ซ้ำ | 7 | 6 |

จำนวน feature ในแต่ละกลุ่ม

| กลุ่ม | LightGBM (perm / SHAP) | LR (perm / SHAP) |
|---|---|---|
| fingerprint only | 10 / 13 | 2 / 12 |
| entangled | 8 / 13 | 16 / 14 |
| attack only | 9 / 5 | 33 / 18 |
| neither | 33 / 29 | 9 / 16 |

สิ่งที่เห็น

- ฝั่ง dataset SHAP พบ feature ที่บอก dataset ได้ 26 ตัว ส่วน permutation พบ 18 ตัว ตรงกัน 13 ตัว SHAP พบเพิ่ม 13 ตัว และไม่นับ 5 ตัวที่ permutation นับ
- LightGBM มี feature ที่ permutation ให้เป็น neither ย้ายไป fingerprint only 6 ตัว และ entangled 3 ตัว ตรงกับที่ทำนายไว้ในเกณฑ์ ส่วนกลุ่ม entangled เดิม 7 จาก 8 ตัวยังอยู่ที่เดิม
- LR ต่างมากเพราะเกณฑ์ 1/60 เข้มกว่าเกณฑ์ noise floor ของ permutation สำหรับ LR feature ที่นับว่ามีสัญญาณการโจมตีลดจาก 39 / 33 ตัว เหลือ 25 / 16 ตัว (2017 / 2018) LR กระจายน้ำหนักไปหลาย feature ส่วนแบ่งแต่ละตัวจึงต่ำกว่า 1/60 ได้ง่าย ความต่างของ LR จึงมาจากเกณฑ์ตัดสินส่วนหนึ่ง ไม่ได้มาจากวิธี SHAP อย่างเดียว

- feature ที่เปลี่ยนกลุ่มประมาณครึ่งหนึ่งมีคู่ที่สหสัมพันธ์สูง เช่น Packet Length Mean กับ Average Packet Size, Fwd IAT Max กับ Idle Max ตรงกับเหตุผลที่ทำ SHAP ตั้งแต่แรก
- หมายเหตุ: การตรวจสหสัมพันธ์ใช้ข้อมูล 2017 ครบ 60 features แต่ไฟล์ 2018 ตรงชื่อแค่ 24 ตัว (ชื่อ column ต่างกัน) ผลส่วนนี้จึงอิง 2017 เป็นหลัก

หมายเหตุ: Part A ใช้การแบ่งสุ่ม 70/30 ภายในของมันเอง ผลจึงไม่เปลี่ยนตามการแก้ split ค่า Spearman ของ LightGBM ฝั่ง 2018 ขยับ 0.683 เป็น 0.687 (ไม่ได้ตรวจสาเหตุ น่าจะเป็นความไม่คงที่ของการเทรน)

## 8. ผล Part B: การโจมตีที่ถูกทายพลาดหลังย้าย dataset

ทิศหลัก 2017 ไป 2018 ค่าเฉลี่ย 5 seeds (s = ส่วนของแรงที่ผลักไปทาง Benign ที่มาจาก feature กลุ่ม fingerprint only และ entangled)

| คลาส | ถูกตัดสินไหม | s(missed) - s(caught) ตาราง SHAP | ตาราง permutation |
|---|---|---|---|
| DDoS | ใช่ (สนับสนุน) | +0.144 (5 จาก 5 seeds >= 0.10) | -0.050 |
| DoS | ใช่ (กลาง ๆ) | +0.080 (2 จาก 5 seeds >= 0.10) | +0.037 |
| Brute Force | ใช่ (ไม่สนับสนุน) | -0.188 | -0.094 |
| Web Attack | ใช่ (ไม่สนับสนุน) | -0.063 | +0.023 |
| Infiltration | ไม่ (caught แค่ 11 แถว) | +0.156 | -0.057 |
| Bot | ไม่ (missed แค่ 5 แถว) | +0.017 | -0.057 |

คำตัดสินตามเกณฑ์

- ใช้ตาราง SHAP: partial (สนับสนุน 1 คลาส, ไม่สนับสนุน 2 คลาส, จาก 4 คลาสที่ตัดสินได้)
- ใช้ตาราง permutation: partial (สนับสนุน 0 คลาส, ไม่สนับสนุน 2 คลาส, จาก 4 คลาสที่ตัดสินได้)
- เนื่องจาก Part A ไม่ยืนยัน ต้องรายงานทั้งสองแบบ ตามที่เขียนไว้ในเกณฑ์

สิ่งที่เห็น

- คำตอบของ Part B ยังไม่ชี้ขาดทั้งสองตาราง จึงยังพูดไม่ได้ว่า fingerprint เป็นสาเหตุที่ทำให้ทายพลาด
- DDoS ในตาราง SHAP ยังเป็นคลาสที่ชัดที่สุด ทุก seed ได้ส่วนต่างเกิน 0.10 (0.103 ถึง 0.180) แต่ในตาราง permutation เป็นลบ
- Brute Force ตอนนี้ตัดสินได้และได้ส่วนต่างเป็นลบในทั้งสองตาราง (SHAP -0.188, permutation -0.094) คือแถวที่ทายพลาดได้แรงจาก feature กลุ่มนี้น้อยกว่าแถวที่ทายถูก ฉบับ 60% ไม่มีแถว caught เพราะ train ไม่มี SSH-Patator
- Web Attack ได้ลบในตาราง SHAP (-0.063) และใกล้ศูนย์ในตาราง permutation (+0.023)
- ค่า s ของตาราง SHAP สูงกว่า (0.6 ถึง 0.8) เพราะกลุ่ม F มี 26 ตัว ส่วนตาราง permutation มี 18 ตัว การเทียบจึงดูที่ส่วนต่างภายในตารางเดียวกันเท่านั้น

ทิศ 2018 ไป 2017 (รายงานไว้ แต่ไม่ใช้ตัดสิน) ส่วนต่างเป็นบวกในหลายคลาส เช่น DDoS +0.322 (ตาราง SHAP) และ +0.054 (ตาราง permutation), Web Attack +0.254 และ +0.265 ทิศนี้ย้าย dataset ได้ดีกว่าอยู่แล้ว ผลนี้น่าสนใจแต่ไม่ได้อยู่ในเกณฑ์ตัดสิน

## 9. ข้อจำกัด

- LightGBM ใช้ SHAP แบบ path-dependent ส่วน LR ใช้แบบ interventional (เหตุผลอยู่ในข้อ 5)

- เกณฑ์ 1/60 ไม่ได้เหมาะกับทุกโมเดลเท่ากัน โมเดลที่กระจายน้ำหนัก เช่น LR จะมี feature ผ่านเกณฑ์น้อยกว่า
- Part B ตัดสินได้แค่ 4 คลาส จาก 6 คลาสการโจมตี (Bot และ Infiltration มีแถวไม่พอ)
- ใช้แค่ LightGBM และ LR ยังไม่ได้ทำ RF, XGBoost, CatBoost

## 10. ขั้นต่อไปที่เป็นไปได้

1. ให้เพื่อนตรวจผลและเกณฑ์ ก่อนนำไปเขียนในรายงานหลัก
2. ขยาย Part A และ B ไป RF, XGBoost, CatBoost ด้วยเกณฑ์เดิม (TreeExplainer เร็ว)
3. ดู DDoS เป็นกรณีศึกษา ว่า feature ไหนผลักไปทาง Benign มากที่สุด
4. นำผลไปใช้กับงาน binary และ distribution alignment ที่ยังค้างอยู่

## 11. ตรวจความทนทาน 1a: เปลี่ยนเกณฑ์ตัดสิน

เป็นการวิเคราะห์เพิ่มเติม ไม่ได้เปลี่ยนคำตัดสินของ Part A สคริปต์ `scripts/shap_threshold_check.py` ใช้ผลที่บันทึกไว้ ไม่ได้เทรนใหม่

Part A รอบ 1 เปลี่ยนสองอย่างพร้อมกัน คือวิธีวัด (SHAP แทน permutation) และเกณฑ์ (1/60 แทน noise floor) ข้อนี้ใช้เกณฑ์ top-K แบบเดียวกันกับทั้งสองวิธี เพื่อดูผลของวิธีวัดอย่างเดียว

| K | ฝั่ง dataset ตรงกัน | kappa LightGBM | kappa LR |
|---|---|---|---|
| 5 | 80% | 0.495 | 0.458 |
| 10 | 77% | 0.448 | 0.411 |
| 15 | 83% | 0.517 | 0.622 |
| 20 | 80% | 0.403 | 0.481 |

เมื่อใช้เกณฑ์เดียวกัน kappa ขึ้นจาก 0.33 ถึง 0.37 เป็น 0.40 ถึง 0.62 ความต่างในรอบ 1 จึงมาจากเกณฑ์ส่วนหนึ่ง แต่สองวิธีก็ยังไม่ตรงกันทั้งหมด (ส่วนใหญ่อยู่ในช่วง "moderate") ค่าที่ K = 5 ขยับเล็กน้อยจากฉบับ 60% (0.524 เป็น 0.495 และ 0.486 เป็น 0.458) ซึ่งเป็นความขยับเล็กน้อยของ Part A ที่ระบุไว้ในหัวข้อ "ผลที่เปลี่ยน" K อื่นเท่าเดิม

## 12. ตรวจความทนทาน 1b: เปลี่ยนโหมด SHAP

สคริปต์ `scripts/shap_mode_check.py` ใช้ LightGBM ของ Part A หนึ่งตัว (2017, seed 42) อธิบาย 300 แถวเดียวกันทั้งสองแบบ (background 100 แถว)

| | ค่า |
|---|---|
| Spearman ของส่วนแบ่ง feature | 0.98 |
| top-10 feature ตรงกัน | 10 จาก 10 |
| ผ่านเกณฑ์ 1/60 | 15 ตัวทั้งสองแบบ ตัดสินตรงกัน 96.7% |
| เวลา | path-dependent 3 วินาที, interventional 127 วินาที |

การเปลี่ยนเป็น path-dependent ในรอบ 1 จึงแทบไม่กระทบผล ข้อจำกัด: ตรวจแค่ 1 โมเดล 1 seed และฝั่ง 2017 เท่านั้น

## 13. Part B รอบ 2: เพิ่ม null จากการสุ่มกลุ่ม feature

เกณฑ์: `results/crossdataset/importance_comparison/SHAP_ROUND2_PREREGISTRATION.md`
เขียนหลังเห็นผลรอบ 1 และระบุไว้ในไฟล์ ผลรอบ 1 ยังเป็นคำตัดสินเดิม ไม่ได้ถูกแทนที่ สคริปต์ `scripts/shap_round2.py` ผลอยู่ที่ `results/crossdataset/importance_comparison/shap_round2/`

สิ่งที่เปลี่ยนจากรอบ 1

- ใช้กลุ่ม F แบบ consensus คือ feature ที่ทั้งสองวิธีเห็นตรงกันว่าบอก dataset ได้ (13 ตัว) ไม่ต้องเลือกว่าจะเชื่อตารางไหน
- สุ่มกลุ่ม feature ขนาดเท่ากัน 2,000 ครั้ง แล้วดูว่าส่วนต่าง G ของ F จริงสูงกว่ากลุ่มสุ่มที่เปอร์เซ็นต์ (ค่า p)
- class ที่สนับสนุนต้องได้ $p < 0.05$ / (จำนวน class ที่ตัดสินได้) ตอนนี้มี 4 class จึงเป็น 0.0125 (ฉบับ 60% มี 3 class เกณฑ์ 0.0167)

โมเดล แถว และการตั้งค่า SHAP เหมือนรอบ 1 ทุกอย่าง ค่า G ของตาราง SHAP และ permutation ในรอบนี้ตรงกับรอบ 1 ทุกคลาสที่ตัดสินได้ (ต่างกันมากสุด 0.0001) ยืนยันว่ารันซ้ำได้ผลเดิม

ทิศหลัก 2017 ไป 2018

| คลาส | กลุ่ม F | G | 95% ของกลุ่มสุ่ม | p | ผล |
|---|---|---|---|---|---|
| DDoS | consensus (13) | -0.012 | 0.169 | 0.56 | ไม่สนับสนุน |
| DoS | consensus (13) | +0.022 | 0.211 | 0.46 | ไม่สนับสนุน |
| Brute Force | consensus (13) | -0.097 | 0.181 | 0.83 | ไม่สนับสนุน |
| Web Attack | consensus (13) | +0.031 | 0.228 | 0.52 | ไม่สนับสนุน |
| DDoS | SHAP (26) | +0.144 | 0.206 | 0.13 | ไม่สนับสนุน |
| DoS | SHAP (26) | +0.080 | 0.238 | 0.29 | ไม่สนับสนุน |
| Brute Force | SHAP (26) | -0.189 | 0.211 | 0.94 | ไม่สนับสนุน |
| Web Attack | SHAP (26) | -0.063 | 0.327 | 0.61 | ไม่สนับสนุน |
| DDoS | perm (18) | -0.050 | 0.191 | 0.65 | ไม่สนับสนุน |
| DoS | perm (18) | +0.037 | 0.228 | 0.39 | ไม่สนับสนุน |
| Brute Force | perm (18) | -0.093 | 0.193 | 0.81 | ไม่สนับสนุน |
| Web Attack | perm (18) | +0.023 | 0.273 | 0.49 | ไม่สนับสนุน |

คำตัดสินตามเกณฑ์: ไม่สนับสนุน (กลุ่ม consensus ซึ่งเป็นกลุ่มหลัก และอีกสองกลุ่มก็ได้ผลเดียวกัน)

สิ่งที่เห็น

- ส่วนต่าง +0.144 ของ DDoS ที่ดูชัดในรอบ 1 อยู่ต่ำกว่าช่วงบนของกลุ่มสุ่ม (0.206) กลุ่ม feature สุ่มขนาดเท่ากันให้ส่วนต่างแบบนี้ได้บ่อย ในรอบ 1 จึงอ่านผลนี้มากเกินไป
- Brute Force เป็นคลาสที่เพิ่มเข้ามา ได้ G ติดลบในทั้งสามกลุ่ม (consensus -0.097) ไม่สนับสนุน
- ช่วงของกลุ่มสุ่มกว้าง (ช่วงบนประมาณ 0.17 ถึง 0.33) แปลว่าแถวที่ทายพลาดต่างจากแถวที่ทายถูกในหลาย feature ไม่ได้กระจุกอยู่ที่ feature ที่บอก dataset
- ข้อสรุป: จาก LightGBM และมุมมองของ SHAP ยังไม่มีหลักฐานว่า feature fingerprint เป็นตัวผลักการโจมตีให้ถูกทายเป็น Benign หลังย้าย dataset ต้องหาคำอธิบายอื่น เช่น การกระจายของค่า feature ที่เลื่อนไปทั้งชุด ซึ่งตรงกับแนวทาง distribution alignment

ทิศ 2018 ไป 2017 (ไม่ใช้ตัดสิน) มี 4 ค่าที่ $p < 0.05$ คือ consensus Bot $p = 0.0235$, consensus Web Attack $p = 0.0305$, SHAP DDoS $p = 0.0080$, perm Bot $p = 0.0415$ แต่เป็นทิศที่ไม่ได้ตั้งเป็นคำถามหลัก และมีการทดสอบหลายครั้ง (18 ค่าในทิศนี้) จึงถือเป็นข้อสังเกตที่ต้องตรวจเพิ่ม ไม่ใช่ข้อค้นพบ

## 14. สรุปรวมทั้งหมด

1. SHAP กับ permutation ไม่ตรงกันพอที่จะยืนยันตารางบทบาทของ feature (Part A) ส่วนหนึ่งเพราะเกณฑ์ ส่วนหนึ่งเพราะ feature ที่ซ้ำและสัมพันธ์กัน
2. ไม่มีหลักฐานว่า feature ที่บอก dataset เป็นสาเหตุที่การโจมตีถูกทายพลาดหลังย้าย dataset (Part B รอบ 1 ขึ้นกับตาราง รอบ 2 ไม่สนับสนุนทุกกลุ่ม)
3. ผลนี้ชี้ว่าการตัด feature fingerprint ทิ้งอย่างเดียวไม่น่าจะแก้ปัญหาการย้าย dataset ได้ ซึ่งสอดคล้องกับผล ablation เดิมที่ตัด feature แล้วไม่ดีขึ้น

## 15. เกณฑ์ที่ต้องผ่านแบบละเอียด (null 200,000 ครั้ง)

เป็นข้อมูลเสริม ไม่ได้เปลี่ยนคำตัดสินของรอบ 2 (คำตัดสินใช้การสุ่ม 2,000 ครั้ง seed 2026 ตามเกณฑ์) สคริปต์ `scripts/shap_null_thresholds.py` ผลอยู่ที่ `shap_round2/null_thresholds.csv`

ขั้นตอน

- ใช้ค่า SHAP รายแถวที่บันทึกไว้จากการรันรอบ 2 บน 70/30 (`shap_round2/blocks.joblib`)
- สุ่มกลุ่ม feature เพิ่มทีละขั้น 2k, 5k, 10k, 20k, 50k, 100k, 200k (seed 2027 แยกจากตัวที่ใช้ตัดสิน) หยุดเมื่อจุด 98.33% เปลี่ยนน้อยกว่า 0.002 ซึ่งถึงที่ 200,000 ครั้ง

จุดที่ต้องผ่านคือ 1 - 0.05/n เมื่อ n คือจำนวนคลาสที่ตัดสินได้ (ตอนนี้ n = 4 จึงเป็น 98.75% ตามคอลัมน์ "ต้องได้" ในตาราง)

ทิศหลัก 2017 ไป 2018

| กลุ่ม F | คลาส | G ที่ได้ | ต้องได้ | ช่วงความเชื่อมั่น 95% | ขาดอีก | p (200k) |
|---|---|---|---|---|---|---|
| consensus (13) | DDoS | -0.012 | 0.230 | 0.229 ถึง 0.232 | 0.242 | 0.56 |
| consensus (13) | DoS | +0.022 | 0.268 | 0.266 ถึง 0.270 | 0.246 | 0.44 |
| consensus (13) | Brute Force | -0.097 | 0.218 | 0.217 ถึง 0.219 | 0.315 | 0.83 |
| consensus (13) | Web Attack | +0.031 | 0.288 | 0.287 ถึง 0.290 | 0.258 | 0.52 |
| SHAP (26) | DDoS | +0.144 | 0.271 | 0.269 ถึง 0.272 | 0.127 | 0.14 |
| SHAP (26) | DoS | +0.080 | 0.325 | 0.323 ถึง 0.327 | 0.245 | 0.30 |
| SHAP (26) | Brute Force | -0.189 | 0.266 | 0.264 ถึง 0.267 | 0.454 | 0.94 |
| SHAP (26) | Web Attack | -0.063 | 0.392 | 0.391 ถึง 0.394 | 0.456 | 0.62 |
| perm (18) | DDoS | -0.050 | 0.253 | 0.252 ถึง 0.254 | 0.303 | 0.67 |
| perm (18) | DoS | +0.037 | 0.299 | 0.297 ถึง 0.300 | 0.262 | 0.40 |

| กลุ่ม F | คลาส | G ที่ได้ | ต้องได้ | ช่วงความเชื่อมั่น 95% | ขาดอีก | p (200k) |
|---|---|---|---|---|---|---|
| perm (18) | Brute Force | -0.093 | 0.240 | 0.239 ถึง 0.241 | 0.334 | 0.80 |
| perm (18) | Web Attack | +0.023 | 0.336 | 0.335 ถึง 0.338 | 0.313 | 0.49 |

สิ่งที่เห็น

- เกณฑ์ที่ต้องผ่านอยู่ที่ประมาณ 0.22 ถึง 0.39 แล้วแต่คลาสและกลุ่ม
- ค่าที่ใกล้ที่สุดคือ SHAP (26) DDoS ได้ +0.144 ต้องได้ 0.271 ขาดอีก 0.127
- ค่า p จากการสุ่ม 200,000 ครั้งใกล้กับค่าจาก 2,000 ครั้งทุกค่า (ต่างกันมากสุด 0.019) คำตัดสินจึงไม่ได้ขึ้นกับจำนวนการสุ่ม
- ข้อสรุปของข้อ 13 และ 14 เหมือนเดิม

## 16. รอบ 3: RF, XGBoost, CatBoost

เกณฑ์: `results/crossdataset/importance_comparison/SHAP_ROUND3_PREREGISTRATION.md` เขียนก่อนรัน สคริปต์ `scripts/shap_round3.py` ผลอยู่ที่ `results/crossdataset/importance_comparison/shap_round3/` ผลรอบ 1 และรอบ 2 ยังเป็นคำตัดสินเดิม

รอบ 2 ใช้แค่ LightGBM รอบนี้ถามว่าผลเดียวกันเกิดกับโมเดลต้นไม้ตัวอื่นใน protocol ด้วยหรือไม่

สิ่งที่เปลี่ยนจากรอบ 2

- ใช้ RF, XGBoost, CatBoost แทน LightGBM ตั้งค่าและเทรนเหมือน `run_crossdataset.py`
- CatBoost ใช้ SHAP ของ CatBoost เอง เพราะ `shap.TreeExplainer` ทำให้โปรแกรมพัง (access violation) ตอนอ่านโมเดล CatBoost
- ค่า SHAP ของ CatBoost รวมกันแล้วตรงกับผลของโมเดล ต่างไม่เกิน 2e-14 (ทดสอบ 200 แถวในรอบที่รันบน 60%)
- อย่างอื่นเหมือนรอบ 2 ทุกอย่าง: กลุ่ม F สามกลุ่ม, สุ่ม 2,000 ครั้ง seed 2026, Bonferroni
- class ที่ตัดสินได้คิดแยกแต่ละโมเดล RF และ CatBoost ตัดสินได้ 5 คลาส XGBoost ได้ 3 คลาส

### Part B (ใช้ตัดสิน)

ทิศหลัก 2017 ไป 2018 กลุ่ม consensus (13) ค่าในวงเล็บคือ p

| โมเดล | DDoS | DoS | Brute Force | Bot | Web Attack | ผล |
|---|---|---|---|---|---|---|
| LightGBM (รอบ 2) | -0.012 (0.56) | +0.022 (0.46) | -0.097 (0.83) | ไม่ตัดสิน | +0.031 (0.52) | ไม่สนับสนุน |
| xgboost | -0.078 (0.81) | +0.125 (0.06) | ไม่ตัดสิน | ไม่ตัดสิน | -0.100 (0.73) | ไม่สนับสนุน |
| catboost | +0.075 (0.22) | +0.188 (0.06) | -0.230 (0.96) | -0.065 (0.77) | -0.120 (0.77) | ไม่สนับสนุน |
| random_forest | -0.194 (0.94) | -0.180 (0.91) | +0.004 (0.59) | -0.314 (0.96) | -0.233 (0.79) | ไม่สนับสนุน |

ผลของทุกกลุ่ม feature

| โมเดล | consensus (13) | SHAP (26) | perm (18) |
|---|---|---|---|
| xgboost | ไม่สนับสนุน | ไม่สนับสนุน (รายคลาสผ่าน: DoS) | ไม่สนับสนุน |
| catboost | ไม่สนับสนุน | ไม่สนับสนุน (รายคลาสผ่าน: DDoS) | ไม่สนับสนุน |
| random_forest | ไม่สนับสนุน | ไม่สนับสนุน | ไม่สนับสนุน |

"ไม่ชัด" คือ p ต่ำกว่า 0.05 แต่ไม่ถึงเกณฑ์ Bonferroni (0.05 / จำนวนคลาสที่ตัดสินได้ของโมเดลนั้น) ส่วน "รายคลาสผ่าน" คือ G > 0 และ p ต่ำกว่าเกณฑ์นั้น

คำตัดสินรอบ 3: ไม่สนับสนุน (3 จาก 3 โมเดล เกณฑ์ต้องการ 2 จาก 3 โดยใช้กลุ่ม consensus)

กรณีที่ผ่านรายคลาสในกลุ่ม SHAP 26 ตัวมี 2 กรณี คือ xgboost DoS (G = +0.228, p = 0.0045) และ catboost DDoS (G = +0.246, p = 0.0090) ทั้งสองไม่ได้อยู่ในกลุ่มหลัก
และไม่ทำให้ภาพรวมของโมเดลนั้นเปลี่ยน (XGBoost ผ่าน 1 จาก 3 คลาส CatBoost ผ่าน 1 จาก 5 คลาส ต้องเกินครึ่ง) กรณี XGBoost DoS ยังถูกตรวจใน within-target แล้ว (รายงาน `shap_within_target.md`) ด้วยกลุ่ม SHAP เดียวกันได้ G_within = +0.073 (p = 0.21) แปลว่าส่วนต่างบวกนั้นมาจากการที่ข้อมูลต่าง dataset ไม่ได้มาจากการทายผิด

### Part A (รายงานอย่างเดียว ไม่ตัดสิน)

ยังไม่มีผล permutation ของ 3 โมเดลนี้ จึงเทียบบทบาทของ feature กับตาราง SHAP ของ LightGBM แทน

| โมเดล | kappa เทียบ LightGBM | Spearman 2017 | Spearman 2018 | feature ที่ใช้แยกการโจมตี | บทบาทต่างจาก LightGBM |
|---|---|---|---|---|---|
| xgboost | 0.876 | 0.885 | 0.905 | 19 | 5 ตัว |
| catboost | 0.770 | 0.866 | 0.857 | 15 | 9 ตัว |
| random_forest | 0.626 | 0.699 | 0.819 | 28 | 16 ตัว |

LightGBM ใช้ 18 feature ในการแยกการโจมตี Part A ไม่ใช้ split ที่แท้ ค่าของ CatBoost และ RF จึงเท่าฉบับ 60% ส่วน XGBoost ขยับ (kappa 0.874 เป็น 0.876 และ feature ที่ใช้แยกการโจมตี 15 เป็น 19 ตัว) ยังไม่ทราบสาเหตุแน่ชัด ไม่ได้ทดสอบ และไม่กระทบคำตัดสินเพราะ Part A รอบ 3 รายงานอย่างเดียว

## สิ่งที่เห็น

- ทั้ง 3 โมเดลในกลุ่มหลักได้ผลเหมือน LightGBM ไม่มีโมเดลไหนที่ feature fingerprint ผลักการโจมตีไปทาง Benign มากกว่ากลุ่ม feature สุ่มขนาดเท่ากัน
- DoS มีค่า G เป็นบวกในโมเดล boosting ทั้งสองตัวที่ตัดสินได้ (xgboost, catboost) สูงสุด CatBoost +0.188 p = 0.063 แต่ไม่มีค่าไหนผ่านเกณฑ์ในกลุ่มหลัก และ RF เป็นลบเกือบทุกคลาส
- Brute Force ตัดสินได้ใน RF และ CatBoost ตอนนี้ ได้ G ใกล้ศูนย์หรือติดลบ ไม่สนับสนุน
- Part A: XGBoost และ CatBoost ใช้ feature ชุดใกล้กับ LightGBM มาก ส่วน RF กระจายน้ำหนักไปหลาย feature กว่า ตรงกับที่คาดไว้
- kappa ของ XGBoost และ CatBoost สูงกว่าที่คาดไว้ก่อนรัน (0.5 ถึง 0.7)
- ทิศ 2018 ไป 2017 (ไม่ใช้ตัดสิน) มี p < 0.05 แค่ 1 จาก 54 ค่า (xgboost SHAP DDoS) ใกล้กับที่ความบังเอิญให้ได้ (ประมาณ 3 ค่า)
- ทิศ 2017 ไป 2018 ที่ตัดสินได้ 39 ค่า มี p < 0.05 2 ค่า (คือสองกรณีรายคลาสข้างต้น) ใกล้เคียงกับที่ความบังเอิญให้ได้ (ราว 2.0 ค่า แม้การทดสอบเหล่านี้ไม่อิสระต่อกัน) และเป็นกลุ่ม SHAP ซึ่งไม่ใช่กลุ่มตัดสินหลัก

### ปัญหาระหว่างรัน (การรันบน 70/30)

• รันขนาน 4 เลนพร้อมกัน (within-target ของ LightGBM และ round 3 ของแต่ละโมเดล) ครั้งแรก RAM ไม่พอทั้ง 4 เลนตอนอ่านข้อมูล 2018 ทั้งชุด (11.4 ล้านแถวก่อนสุ่ม) จึงเปิดใหม่แบบเหลื่อมเวลาเลนละ 5 นาที ผลไม่ถูกเขียนทับเพราะสคริปต์ไม่รวมผลเมื่อมีเลนล้ม

• เลนแต่ละโมเดลเขียนผลลงโฟลเดอร์แยก แล้วรวมด้วยโหมด `--cached` ของสคริปต์เดิม ซึ่งคำนวณตารางทั้งหมดใหม่จากค่า SHAP ที่บันทึกไว้ ผลจึงเหมือนรันต่อเนื่อง ล็อกของแต่ละเลนเก็บไว้เป็น `run_<model>.log`

• within-target ของ RF แยกเป็น 10 งาน (ทิศ x seed) เรียกฟังก์ชัน `collect` ของสคริปต์เดิมโดยไม่แก้โค้ดคำนวณ แล้วรวมบล็อกตามลำดับเดิม

• สคริปต์รวมงานที่ผมเขียนมีจุดรอกันเอง (งานสุดท้ายรอไฟล์ที่ถูกคัดลอกหลังงานทั้งหมดเสร็จ) ทำให้เสียเวลารอเปล่าราว 7 ชั่วโมง แก้แล้ว ไม่กระทบผล

• RF ช้าที่สุด ทิศ 2018 ไป 2017 ใช้ประมาณ 80 นาทีต่อ seed เพราะต้นไม้ที่เทรนจาก 2018 ใหญ่ รวมเลน RF ประมาณ 650 นาที

### ข้อจำกัด

• Part A ไม่มี permutation ของ 3 โมเดลนี้ จึงเป็นแค่การเทียบกับ LightGBM ไม่ใช่การยืนยันตาราง

• CatBoost ใช้ SHAP คนละตัวกับโมเดลอื่น (ค่าเป็น tree SHAP แบบเดียวกัน แต่เป็นโค้ดของ CatBoost)

• ยังไม่ได้ทำ MLP, LR และ stacking ใน Part B

## 17. สรุปหลังรอบ 3

1. โมเดลต้นไม้ทั้ง 4 ตัว (LightGBM, XGBoost, CatBoost, RF) ไม่มีหลักฐานว่า feature ที่บอก dataset เป็นตัวผลักการโจมตีให้ถูกทายเป็น Benign หลังย้าย dataset ในกลุ่มหลัก (consensus) กลุ่ม SHAP 26 ตัวมี 2 กรณีรายคลาสที่ผ่านเกณฑ์ แต่ภาพรวมของโมเดลยังไม่สนับสนุน และ within-target ไม่ผ่าน

2. ผลนี้ตรงกับ ablation เดิมที่ตัด feature fingerprint ออกแล้วไม่ดีขึ้น การตัด feature อย่างเดียวจึงไม่น่าจะแก้ปัญหาได้

3. คำอธิบายที่เหลือคือค่าของ feature เลื่อนไปทั้งชุด ขั้นต่อไปจึงเป็น binary mode และ distribution alignment

4. ไม่วางแผนทำ SHAP รอบต่อไป ค่า p ต่ำในทิศ 2018 ไป 2017 เก็บไว้เป็นข้อสังเกตเท่านั้น

5. การเปลี่ยนจาก train 60% เป็น 70% ไม่เปลี่ยนข้อสรุปข้างต้น ที่เปลี่ยนคือจำนวนคลาสที่ตัดสินได้ (Brute Force เพิ่ม) เพราะ SSH-Patator เข้ามาอยู่ใน train ของ 2017